

SIH 2026 — ML & Privacy Module

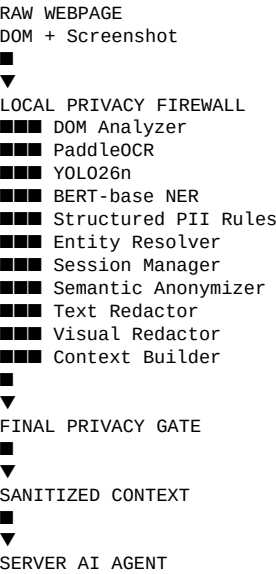
Features, Technology Stack & System Design — PS ID 26171

Module owner: ML Engineer / Browser-side Privacy & Perception

Objective: transform raw DOM + screenshot data into a privacy-safe, semantically useful context for the server-side AI agent. Raw PII and the anonymization mapping remain local to the browser.

1. Scope of My Part

Your module is effectively a **local privacy firewall**. It sits between the webpage and the remote AI agent and owns perception, PII detection, privacy fusion, anonymization, redaction, session identity and sanitized-context delivery.



2. Complete Feature Set

Feature	What it does	Primary technology
DOM analysis	Reads semantic HTML signals, labels, input types, autocomplete, ARIA and coordinates.	JavaScript DOM APIs, MutationObserver
Screenshot perception	Receives/captures visual page state for local processing.	Browser Extension APIs
OCR	Extracts visible text, geometry and confidence from screenshots.	PaddleOCR.js / PP-OCrv5
Privacy-object detection	Detects faces, cards, IDs, passports, signatures, QR/barcodes, etc.	YOLO26n + ONNX
Structured PII detection	Detects email, phone, PAN, Aadhaar, card candidates, IP/URL, OTP etc.	Regex + validation + DOM rules
Contextual PII detection	Detects PERSON, LOCATION, ORGANIZATION and similar semantic entities.	BERT-base NER
Entity fusion	Merges overlapping DOM/OCR/CV/NER detections into one entity.	Custom geometry + confidence fusion
Session identity	Maintains stable anonymous identity across observations.	JS Map + crypto.getRandomValues
Semantic anonymization	Replaces sensitive values with labels such as [PERSON_A71C].	Custom SemanticAnonymizer
Visual redaction	Masks sensitive pixels before network transmission.	Canvas/ImageBitmap processing
Text redaction	Replaces sensitive OCR/DOM spans while preserving surrounding semantics.	Custom span redactor
Sanitized context	Builds structured context for the agent.	JSON schema / ContextBuilder
Privacy gate	Blocks transmission if sensitive information may remain.	Custom validation/audit layer
Performance isolation	Runs ML inference without blocking the webpage.	Web Worker
Browser inference	Executes models locally.	ONNX Runtime Web / Transformers.js
Hardware acceleration	Uses GPU where possible with broad fallback.	WebGPU + WASM
Evaluation	Measures PII recall/precision, redaction, latency and resource use.	Custom benchmark/evaluation suite

3. Feature 1 — DOM Analyzer

The DOM analyzer is the first privacy signal because HTML often explicitly identifies sensitive fields even when visual models cannot.

```
<input type="password" autocomplete="current-password">  
<label>Email</label><input type="text">
```

↓

```
PASSWORD → confidence 0.99  
EMAIL → confidence 0.95  
bbox → getBoundingClientRect()
```

- Inspect type, name, id, placeholder, aria-label, autocomplete, labels and nearby semantic context.
- Extract bounding boxes with `getBoundingClientRect()`.
- Ignore irrelevant hidden nodes.
- Use `MutationObserver` for dynamically changing SPA pages.
- Emit normalized Detection objects to the Privacy Engine.

4. Feature 2 — PaddleOCR

PaddleOCR is the primary OCR engine. It runs locally and supplies the text geometry needed to connect OCR output to PII detection and screenshot redaction.

```
Screenshot  
↓  
PaddleOCR.js  
↓  
text + polygon/bbox + confidence  
↓  
PII rules / BERT NER  
↓  
sensitive text spans  
↓  
redaction + semantic labels
```

5. Feature 3 — YOLO26n Privacy Detector

YOLO26n provides visual detection of privacy-sensitive objects that OCR and DOM analysis cannot reliably identify.

Initial classes: FACE, ID_CARD, PASSPORT, CREDIT_CARD, SIGNATURE, QR_CODE, BARCODE, and optionally DRIVER_LICENSE.

- Fine-tune on a privacy-specific dataset.
- Export to ONNX for browser execution.
- Start with FP32 browser baseline, then benchmark FP16/INT8.
- Return class, bounding box, confidence and source.
- Use conservative padding around high-risk regions before redaction.

6. Feature 4 — BERT-base NER

BERT-base NER supplies contextual understanding for entities such as PERSON, LOCATION and ORGANIZATION. It complements deterministic PII rules rather than replacing them.

```
OCR:  
"John Smith lives in New Delhi"  
  
BERT-base NER  
↓  
PERSON: John Smith  
LOCATION: New Delhi  
↓  
map token spans → OCR boxes  
↓  
Privacy Entity
```

7. Feature 5 — Layered PII Detection

```
■■■ DOM semantic rules  
■
```

OCR text ████████████████████ Regex + validation
█
████ BERT-base NER
↓
PII detections
↓
Entity Resolver

- DOM: PASSWORD, EMAIL, PHONE, OTP, payment fields, credentials.
- Regex: email, phone, PAN, Aadhaar, card candidates, IP, URL and project-specific identifiers.
- NER: PERSON, LOCATION, ORGANIZATION and contextual entities.
- Combine evidence instead of relying on a single detector.

8. Feature 6 — Entity Resolver / Privacy Fusion

The same sensitive entity may be detected by multiple components. The resolver merges these detections using geometry, type compatibility, text similarity and confidence.

```
DOM → EMAIL bbox [100,200,350,235]
OCR → email text bbox [105,202,345,233]
NER → none
```

↓ IoU / overlap

```
ENTITY
type: EMAIL
bbox: [100,200,350,235]
sources: DOM + OCR
```

9. Feature 7 — Session Manager

The session manager creates stable, random semantic identities. Stability matters because the remote agent may observe the same entity over multiple agent iterations.

```
Local mapping:
PERSON:John Smith → [PERSON_A71C]
EMAIL:john@gmail.com → [EMAIL_92F4]
PHONE:+919876543210 → [PHONE_C821]

Server sees:
[PERSON_A71C], [EMAIL_92F4], [PHONE_C821]
```

Server NEVER sees the mapping.

- Use `crypto.getRandomValues()` for identifiers.
- Normalize entity values before lookup.
- Keep a stable token for the same entity during a session.
- Never reuse a token for another entity.
- Provide session reset/expiry.
- Do not log raw values or mappings.

10. Feature 8 — Semantic Anonymizer

The semantic anonymizer preserves meaning while removing the private value.

```
Original:
Welcome John Smith
Email: john.smith@gmail.com
Phone: +91 9876543210
```

```
Sanitized:
Welcome [PERSON_A71C]
Email: [EMAIL_92F4]
Phone: [PHONE_C821]
```

Key behavior: replace only sensitive spans. Keep public UI text, labels, ordering and layout information so the agent can still reason about the page.

11. Feature 9 — Visual Redactor

- FACE → blur/pixelate or opaque mask.
- PASSWORD/SECRET → opaque mask.
- CREDIT_CARD / ID / PASSPORT / SIGNATURE → opaque mask with padding.
- QR_CODE / BARCODE → mask the full code region.
- Perform redaction locally before creating the network payload.

12. Feature 10 — Sanitized Context Builder

```
{
  "session_id": "opaque-id",
  "screen": {"width": 1440, "height": 900},
  "elements": [...],
  "entities": [
```

```

{"type": "PERSON", "label": "[PERSON_A71C]", "bbox": [...]}
],
"screenshot": "<sanitized image>"
}

```

The context should retain actionable UI semantics: roles, visible non-sensitive labels, element geometry, page state and sanitized entities. Avoid transmitting the entire raw DOM when a compact representation is sufficient.

13. Feature 11 — Final Privacy Gate

```

Raw DOM ██████████ X
Raw OCR ██████████ X
Raw PII ██████████ X
Session mapping ██████ X

Sanitized screenshot
+ sanitized DOM
+ semantic entities
↓
PRIVACY AUDIT
↓
NETWORK SEND

```

- Check for raw sensitive spans before serialization.
- Ensure every high-risk visual entity has a mask.
- Ensure session mapping is not serialized.
- Fail closed if privacy analysis is incomplete.
- Optionally run post-redaction OCR during evaluation to measure leakage.

14. Feature 12 — Web Worker Execution

```

Content Script
■
■ postMessage
▼
Inference Worker
■■■ PaddleOCR
■■■ YOLO26n
■■■ BERT NER
■■■ Privacy Engine
■
■ sanitizedContext
▼
Background / Content Script
■
▼
Server Agent

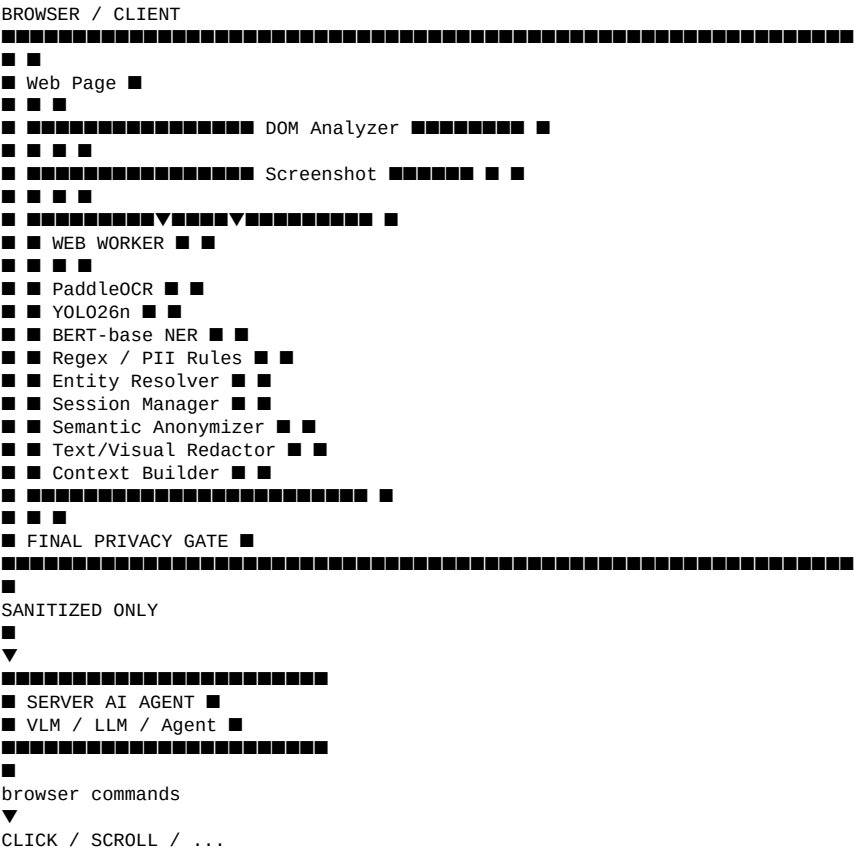
```

This keeps heavy local inference off the page's main execution path.

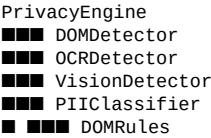
15. Technology Stack

Layer	Technology	Role
Browser extension	Chrome Extension / Firefox WebExtension APIs	Page integration, DOM/screenshot access, messaging
Frontend language	JavaScript / TypeScript	Extension + privacy logic
DOM	DOM APIs + MutationObserver	Semantic page analysis
OCR	PaddleOCR.js / PP-OCrv5	Local text recognition
CV	YOLO26n	Visual privacy-object detection
Model format	ONNX	Portable browser inference models
NER	BERT-base NER	Contextual entity recognition
Inference runtime	ONNX Runtime Web	Local ONNX execution
NLP runtime	Transformers.js	Browser-friendly NER pipeline
Acceleration	WebGPU	GPU inference when supported
Fallback	WASM	Cross-browser inference fallback
Concurrency	Web Workers	Non-blocking local inference
Image processing	Canvas / ImageBitmap APIs	Cropping, masking and redaction
Storage	In-memory session state; extension-local storage if required	Session/token management
Communication	postMessage + HTTPS/REST/WebSocket as selected by team	Worker and server communication
Server handoff	Sanitized JSON + sanitized image	Remote AI agent input

16. Complete System Design



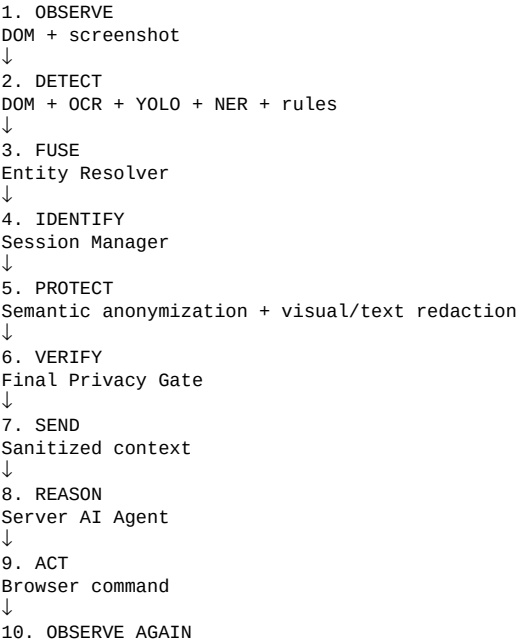
17. Internal Component Design



- ■■■■ RegexRules
- ■■■■ BERTNER
- EntityResolver
- SessionManager
- TextRedactor
- VisualRedactor
- ContextBuilder

Each component should expose a small interface. This makes model replacement and unit testing straightforward.

18. Data Flow



19. Module-to-SIH Evaluation Mapping

SIH evaluation dimension	Your contribution	Evidence to collect
Visual context accuracy	YOLO26n + OCR + DOM geometry	Detection precision/recall, mAP/IoU, OCR text/box quality
PII detection precision/recall	DOM + regex + BERT NER + fusion	Per-class precision, recall, F1
Redaction precision	Semantic + visual redactors	Mask IoU, false-redaction area, leakage tests
Client resource utilization	Worker + ONNX/WebGPU/WASM optimization	RAM, CPU/GPU, model size, inference time
End-to-end latency	Entire local privacy pipeline	Per-stage latency and total time

20. Recommended Implementation Order

1. Shared Detection / Entity / Context schemas
2. DOM Analyzer
3. Regex + validation PII layer
4. PaddleOCR.js
5. YOLO26n ONNX
6. BERT-base NER
7. Coordinate normalization
8. Entity Resolver
9. Session Manager
10. Semantic Anonymizer
11. Text + Visual Redactors
12. Context Builder
13. Final Privacy Gate
14. Web Worker integration
15. Server-agent handoff
16. OBSERVE → PROTECT → SEND → ACT loop
17. Benchmark + optimize

21. Final Deliverable

Your completed module should be demonstrable as a self-contained local privacy firewall: **DOM + screenshot → local perception → PII/entity fusion → session-stable anonymization → visual/text redaction → privacy audit → sanitized context → AI agent.**

Non-negotiable security property: the remote agent receives enough semantic information to perform browser reasoning, but it must never receive the actual sensitive value or the browser-local mapping that could recover it.